OPERATING SYSTEMS AND SYSTEM PROGRAMMING

PRACTICAL – WEEK 5

Name: S Svara Haasini

Roll No: 2520090170

Task 1: Producer-Consumer Communication Using Anonymous Pipe

Question:

Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

Source Code:

```
haasini@SSH:~/OSSP_upload/Practical_Week5$ cat > P5_T1.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>
#include <sys/time.h>

int main(void)
{
    int pipefd[2];
    pid_t pid;
    char message[] = "Operating Systems - Producer Consumer using Pipe";
    char buffer[100];

    struct timeval start, end;

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
EOF return 0;f("Producer and consumer communication completed.\n");,
haasini@SSH:~/OSSP_upload/Practical_Week5$ gcc -Wall -Wextra P5_T1.c -o P5_T1
```

Output:

Producer-Consumer Pipe Execution

```
haasini@SSH:~/OSSP_upload/Practical_Week5$ gcc -Wall -Wextra P5_T1.c -o P5_T1
haasini@SSH:~/OSSP_upload/Practical_Week5$ ./P5_T1
=== PARENT PROCESS: PRODUCER ===
Parent PID : 537
Child PID  : 538
Data sent through pipe:
Operating Systems - Producer Consumer using Pipe
Bytes produced: 48
Pipe communication write time: 6.00 microseconds
=== CHILD PROCESS: CONSUMER ===
Child PID  : 538
Parent PID : 537
Data received from pipe:
Operating Systems - Producer Consumer using Pipe
Bytes consumed: 48
Producer and consumer communication completed.
```

Observations:

1. pipe() creates a unidirectional communication channel with a read end (pipefd[0]) and a write end (pipefd[1]).

2. fork() creates a child process (PID 538) from the parent process (PID 537), with both initially sharing access to the pipe file descriptors.

3. The parent process acts as the producer and writes the message "Operating Systems - Producer Consumer using Pipe" (48 bytes) into the pipe using write().

4. The child process acts as the consumer and reads the same data (48 bytes) from the pipe using read().

5. Unused pipe ends are closed in each process to prevent unnecessary file descriptor leakage and to enable proper EOF detection when the writer closes the pipe.

6. The pipe communication write time measured via gettimeofday() is 6.00 microseconds, representing the time taken for the parent to write 48 bytes into the pipe.

7. The parent waits for the child process using wait() to ensure the child completes before the parent terminates.

8. The equal number of bytes produced (48) and consumed (48) confirms successful data transfer through the anonymous pipe.

Task 2: Implement ls -l | grep ".c" Using System Calls

Question:

Develop a program that executes the equivalent of the shell command: ls -l | grep ".c" using fork(), pipe(), dup2(), and exec() system calls.

Source Code:

```
haasini@SSH:~/OSSP_upload/Practical_Week5$ cat > P5_T2.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void)
{
    int pipefd[2];

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    pid_t ls_pid = fork();

    if (ls_pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (ls_pid == 0)
    {
        /* First child executes: ls -l */

EOF return 0;quivalent shell command: ls -l | grep \".c\"\n");
haasini@SSH:~/OSSP_upload/Practical_Week5$ gcc -Wall -Wextra P5_T2.c -o P5_T2
```

Output:

Shell Pipeline Output & C Pipeline Implementation

```
haasini@SSH:~/OSSP_upload/Practical_Week5$ ls -l | grep ".c"
-rw-r--r-- 1 haasini haasini  2016 Aug 22 03:23 P5_T1.c
-rw-r--r-- 1 haasini haasini  1416 Aug 22 03:24 P5_T2.c
haasini@SSH:~/OSSP_upload/Practical_Week5$ ./P5_T2
-rw-r--r-- 1 haasini haasini  2016 Aug 22 03:23 P5_T1.c
-rw-r--r-- 1 haasini haasini  1416 Aug 22 03:24 P5_T2.c

Pipeline completed successfully.
Equivalent shell command: ls -l | grep ".c"
```

Observations:

1. pipe() creates a unidirectional communication channel between the two processes that will execute ls and grep.

2. fork() is used to create two child processes: one to execute ls -l and another to execute grep ".c".

3. The first child process executes the ls -l command using execlp(), which lists all files and directories with detailed information.

4. The second child process executes the grep ".c" command using execlp(), which filters lines containing the pattern ".c".

5. dup2() redirects the standard output (file descriptor 1) of the ls -l process to the write end of the pipe, ensuring its output goes into the pipe.

6. dup2() redirects the standard input (file descriptor 0) of the grep process to the read end of the pipe, allowing it to receive data from the pipe.

7. The parent process closes its own copy of both pipe file descriptors after the children are created, leaving only the children with active access.

8. The parent waits for both child processes using waitpid(), ensuring synchronization and proper cleanup before the parent terminates.

9. The resulting data flow follows: ls -l → pipe → grep ".c" → terminal, producing output equivalent to the shell pipeline ls -l | grep ".c".